AWS CloudWatch Monitoring

EC2 CPU Alarm & SNS Email Notification

Author	Date	Region	Account
Venkaiah Malli	06 March 2026	eu-west-1 (Ireland)	251622303328
Project Type	**Cost**	**Instance**	**GitHub**
Cloud Monitoring	€0.00 - Free Tier	t3.micro - Amazon Linux 2023	github.com/venkaiahmalli96

Pipeline Overview

EC2 Instance	→	CloudWatch Alarm	→	SNS Topic	→	Email Alert
Cloudwatch-Demo t3.micro		CPU-High_Alert CPU > 50% | 1 min		cloudwatch-alarm-topic		Gmail venkaiahmalli96

Contents

1. **Introduction to AWS CloudWatch**
2. **Key Concepts & Services**
3. **Project Architecture**
4. **Step-by-Step Implementation**
   - Step 1 - Launch EC2 Instance
   - Step 2 - Create SNS Topic & Email Subscription
   - Step 3 - Create CloudWatch Alarm
   - Step 4 - Connect to EC2 & Run Stress Test
   - Step 5 - Alarm Triggered (IN ALARM State)
   - Step 6 - Email Notification Received
   - Step 7 - Resource Cleanup
5. **Project Results & Skills Demonstrated**

1. Introduction to AWS CloudWatch

CloudWatch is the built-in AWS monitoring service. Every time you launch an EC2 instance, a Lambda function, or any other AWS resource, CloudWatch starts collecting data from it automatically. You do not need to install anything or do any extra setup for the basics.

In my day-to-day cloud support work, CloudWatch is usually the first place I check when something looks wrong. If a server is slow, if an application went down overnight, or if a Lambda function is throwing errors - the data is sitting there in CloudWatch. And with alarms set up, it will send you an email the moment something crosses a threshold, so you do not have to keep staring at graphs all day.

Core Capabilities

Monitoring

CloudWatch collects performance data from EC2, RDS, ELB, S3, Lambda, and most other AWS services without you having to configure anything. Metrics like CPU, network, and disk I/O are recorded every minute and stored for 15 months.

Alarms

You pick a metric, set a threshold, and CloudWatch watches it for you. When the value crosses that threshold - say CPU goes above 50% - the alarm fires. It can send an email, reboot the instance, or kick off an auto scaling event.

Logs

You can push log files from EC2, Lambda, RDS, and your own applications into CloudWatch Logs. Once they are there, you can search through them using Logs Insights - much easier than SSH-ing into a server to read log files manually.

Dashboards

CloudWatch lets you build a custom dashboard that puts all your key metrics on one screen. I find this useful when I want a quick health overview without clicking through each service.

Scaling

CloudWatch alarms can trigger Auto Scaling actions. For example, you can set it to add more EC2 instances when CPU stays above 70%, and remove them when it drops back below 30%.

AWS Services CloudWatch Monitors

AWS Service	Key Metrics Monitored
Amazon EC2	CPUUtilization, NetworkIn/Out, DiskReadOps, DiskWriteOps, StatusCheckFailed
Amazon EBS	VolumeReadOps, VolumeWriteOps, VolumeThroughputPercentage, BurstBalance
Elastic Load Balancer	RequestCount, TargetResponseTime, HTTPCode_5XX, HealthyHostCount
Amazon S3	BucketSizeBytes, NumberOfObjects, AllRequests, 4xxErrors
Amazon RDS	CPUUtilization, DatabaseConnections, FreeStorageSpace, ReadLatency
AWS Lambda	Invocations, Duration, Errors, Throttles, ConcurrentExecutions

2. Key Concepts & Services

This project uses four AWS services. Before jumping into the steps, here is a quick explanation of what each one does and why I used it in this setup.

Amazon EC2 - Elastic Compute Cloud

EC2 is how you get a virtual server on AWS. You pick an OS, choose the instance size, and it is running within a couple of minutes. In this project, the EC2 instance is the resource I am monitoring. I deliberately push its CPU above 50% to test whether the CloudWatch alarm actually fires and sends the email.

Instance Type	t3.micro - 2 vCPUs and 1 GB RAM. Free Tier eligible in eu-west-1 and more than enough for running a stress test.
AMI	I used Amazon Linux 2023, which is the latest AWS-maintained Linux version. It comes with yum already set up, so installing packages is straightforward.
Key Pair	I created a new key pair called cloudwatch-key. The .pem file downloads once - I kept it in my Downloads folder for EC2 Instance Connect.
Security Group	I created a new security group and set SSH access to my IP only. Good habit to keep - never open SSH to the whole internet.
EC2 Instance Connect	This lets you open a terminal directly in the browser. No PuTTY, no local SSH setup needed. Really convenient for quick tasks.
Public IP	I enabled auto-assign public IP. The instance got 3.252.203.81 - required for EC2 Instance Connect to work from the console.

Amazon CloudWatch Alarms

A CloudWatch alarm watches one metric and takes action when it crosses a value you set. Without this, you would have to manually open CloudWatch and stare at graphs to catch problems. With an alarm configured, it does the watching for you - even at 3am. The moment CPU crosses 50%, it fires automatically and sends the notification.

Metric	CPUUtilization - the percentage of the EC2 CPU being used at any point in time.
Namespace	AWS/EC2 - this is how CloudWatch groups all EC2-related metrics together.
Period	I changed this from the default 5 minutes to 1 minute. That way the alarm reacts faster when CPU spikes.
Statistic	I chose Average - the mean CPU across all readings in that 1-minute window. A single spike does not immediately fire the alarm.
Threshold	I set it to 50. I chose this value because it is easy to cross during a stress test, and low enough to verify the alarm quickly.

Datapoints to Alarm	1 out of 1 - so it fires after the very first breach. In a production environment I would use 3 out of 3 to reduce false alarms.
Alarm States	INSUFFICIENT DATA when first created (no data yet), OK when CPU is below 50%, and IN ALARM when the threshold is crossed.

Amazon SNS - Simple Notification Service

SNS is the service that actually sends the notification. When the alarm fires, CloudWatch publishes a message to an SNS topic. SNS then delivers that message to everyone subscribed. In this case I subscribed my email address, so I get the alert in Gmail. You can add more subscribers to the same topic - SMS, another email, a Lambda - and they all receive it at the same time without changing the alarm at all.

Topic	I created cloudwatch-alarm-topic as a Standard topic. Standard is fine for notifications - it supports email, SMS, Lambda, and other protocols.
Standard vs FIFO	Standard topics are best for alerts and notifications where the order of messages does not matter. FIFO is for things like payment queues where order is critical.
Subscription	I added my email as a subscriber using the Email protocol. SNS takes the alarm data and formats it into a readable email automatically.
Confirmation	After subscribing, AWS sends a confirmation email. I had to click the link in it before any alarm notifications would actually arrive. Easy to miss this step.
Publisher	The CloudWatch alarm publishes to the topic when CPU crosses 50%. No code needed - I just selected the topic in the alarm configuration.
Subscriber	My Gmail address venkaiahmalli96@gmail.com is the subscriber. I could add more addresses or a phone number to the same topic later if needed.

Linux Stress Tool

stress is a Linux tool that artificially loads the CPU. I used it here to push the instance above 50% so I could verify the alarm actually works end to end. Before you rely on any monitoring alert in a real environment, you want to test it manually at least once - this is exactly that kind of test.

Installation	sudo yum install stress -y - the -y flag just means yes to everything so it installs without asking.
-c 2	2 CPU workers - one for each vCPU on the t3.micro. Each worker maxes out a core, so both vCPUs hit 100%.
-t 300	300 seconds - I ran it for 5 minutes so CloudWatch had time to detect the spike and the alarm stayed in the fired state long enough to confirm.

What happened	CPU jumped to 99.93% almost immediately. CloudWatch caught it on the next 1-minute check and the alarm moved to IN ALARM.
Why this matters	This simulates what would happen if a real application was using all the CPU - a runaway process, a traffic spike, or a memory leak forcing heavy swapping.

3. Project Architecture

The setup is pretty simple once you see the full picture. EC2 pushes CPU data to CloudWatch every minute on its own. CloudWatch checks that data against the 50% threshold. When it crosses, the alarm fires and CloudWatch tells SNS to send the email. I did not write a single line of code - everything was configured through the AWS Console.

Data Flow

EC2 sends CPU data

EC2 automatically sends CPUUtilization to CloudWatch every 60 seconds. I did not need to configure anything for this - it just works.

CloudWatch checks the value

Every minute, CloudWatch looks at the average CPU and compares it to 50%. If it is higher, the alarm state changes.

Alarm fires

The moment the threshold is crossed, the alarm moves from OK to IN ALARM and immediately triggers the SNS notification.

SNS sends the message

CloudWatch publishes to cloudwatch-alarm-topic. SNS picks it up and delivers the email with all the alarm details included.

Email arrives in Gmail

The email landed in my Gmail within about a minute of the alarm firing. It showed the CPU value, timestamp, and a link to the alarm in the console.

Full Configuration

EC2 Instance Name	Cloudwatch-Demo
Instance ID	i-006955522965f21a7
Instance Type	t3.micro - 2 vCPU, 1 GB RAM - Free Tier
AMI	Amazon Linux 2023 (al2023-ami-2023.10.20260216.1-kernel-6.1-x86_64)
Public IP	3.252.203.81
Region	eu-west-1 (Europe - Ireland)
CloudWatch Alarm	CPU-High_Alert
Alarm Condition	CPUUtilization > 50 - Average - 1 minute period - 1 out of 1 datapoints
SNS Topic	cloudwatch-alarm-topic - Standard type
Notification Endpoint	venkaiahmalli96@gmail.com (Email protocol - Confirmed)
Stress Command	stress -c 2 -t 300 (2 CPU workers, 300 seconds)
Total Cost	€0.00 - all resources within AWS Free Tier limits

4. Step-by-Step Implementation

Below is a walkthrough of everything I did to set this up, with the exact settings I used and screenshots from the live AWS Console. I did all of this on 06 March 2026 in the eu-west-1 Ireland region.

1 Launch EC2 Instance

Create and launch a Free Tier virtual server to monitor with CloudWatch

First I launched the EC2 instance that I want to monitor. I went with a t3.micro on Amazon Linux 2023 - Free Tier, no cost. I made sure to enable a public IP because I needed that for EC2 Instance Connect later when installing the stress tool.

Instance Configuration

Name	Cloudwatch-Demo
AMI	Amazon Linux 2023 - Latest version with kernel 6.1
Instance Type	t3.micro - 2 vCPUs, 1 GB RAM - Free Tier eligible
Key Pair	cloudwatch-key - RSA - .pem file (downloaded to local Downloads folder)
VPC	Default VPC - default subnets in eu-west-1
Auto-assign Public IP	Enabled - required for EC2 Instance Connect
Security Group	New - Allow SSH (TCP port 22) from My IP address only
Storage	Default 8 GB gp3 SSD - no changes required

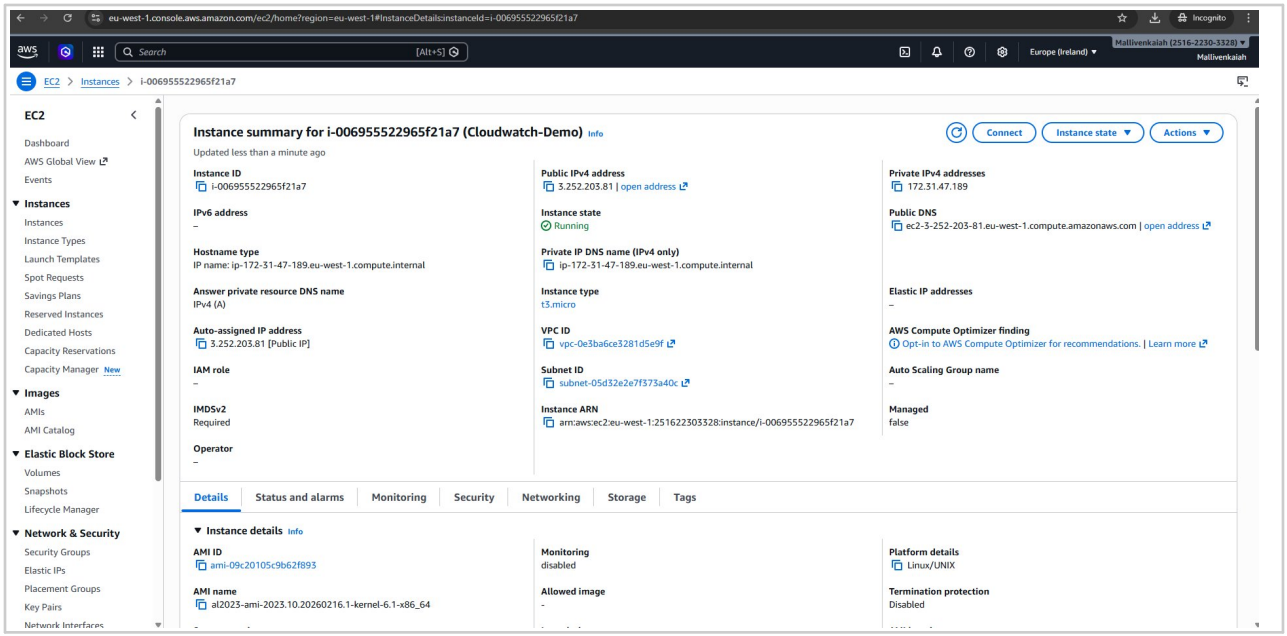


Figure 1.1 - EC2 instance Cloudwatch-Demo in Running state - Instance ID: i-006955522965f21a7

2 Create SNS Topic & Email Subscription

Set up Amazon SNS to deliver email notifications when the alarm fires

Before creating the alarm, I set up the SNS topic first. You need somewhere to send the notification before you can point the alarm at it. I also made sure the email subscription was confirmed straight away - if you leave it in pending state, alarm emails just silently never arrive.

SNS Topic Settings

Topic Type	Standard - supports Email, HTTP/S, Lambda, SQS, SMS, and Mobile Push protocols
Topic Name	cloudwatch-alarm-topic
All other settings	Left as default - no encryption, access policy, or delivery policy required

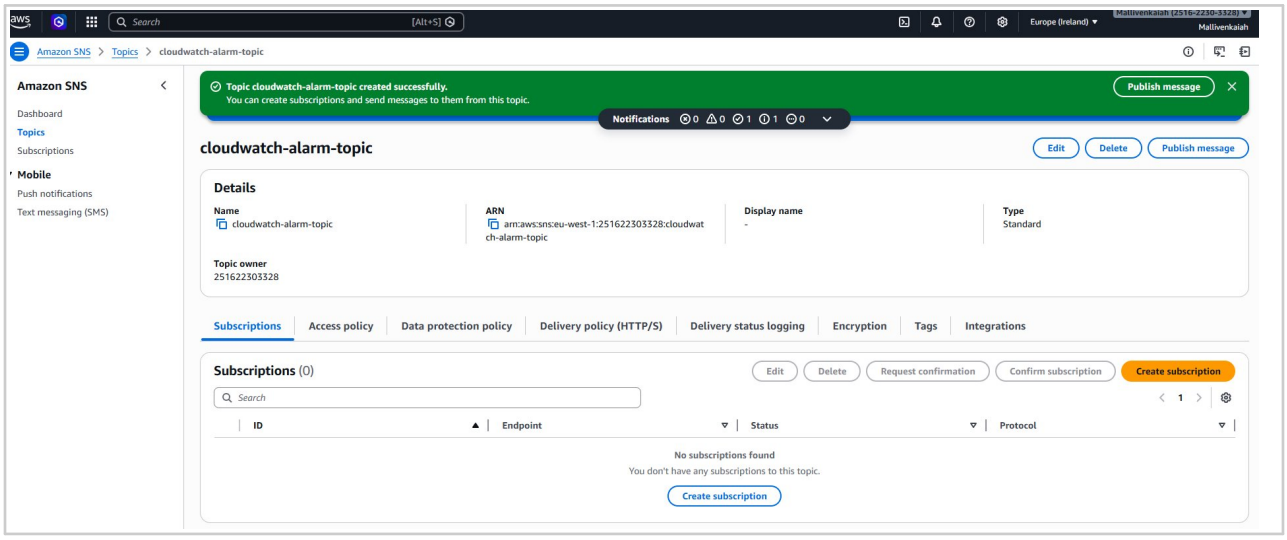


Figure 2.1 - SNS topic cloudwatch-alarm-topic created successfully

Email Subscription

Protocol	Email
Endpoint	venkaiahmalli96@gmail.com

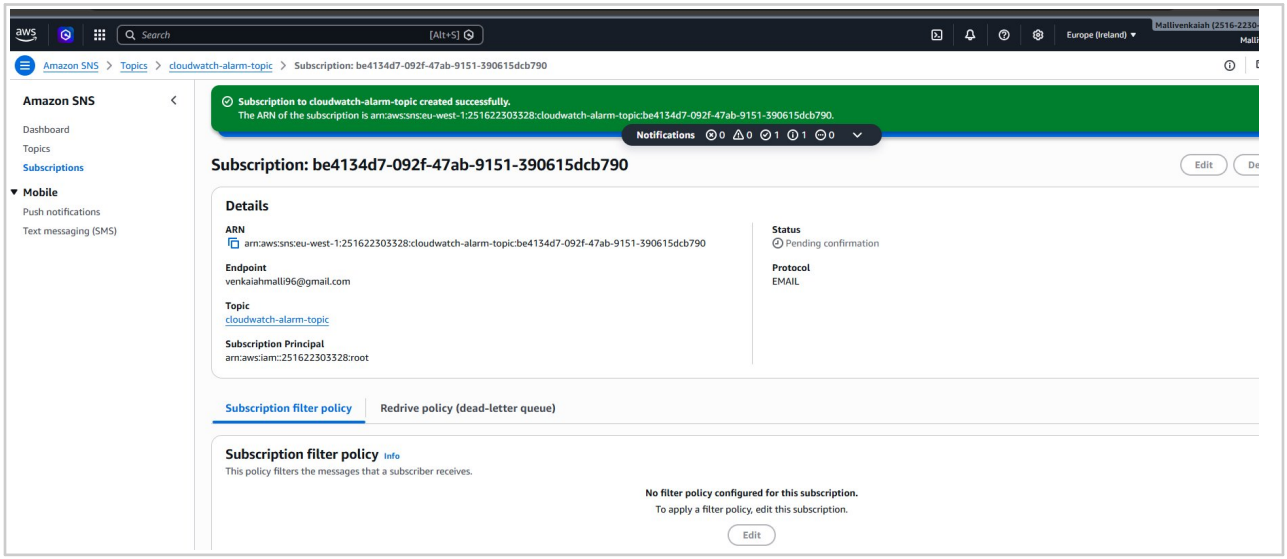


Figure 2.2 - SNS subscription created - Status: Pending confirmation

Confirm Subscription

AWS sent a confirmation email to my Gmail almost immediately. I opened it and clicked the Confirm subscription link. Until you do that, the subscription stays in Pending state and you will not receive any alarm notifications at all.

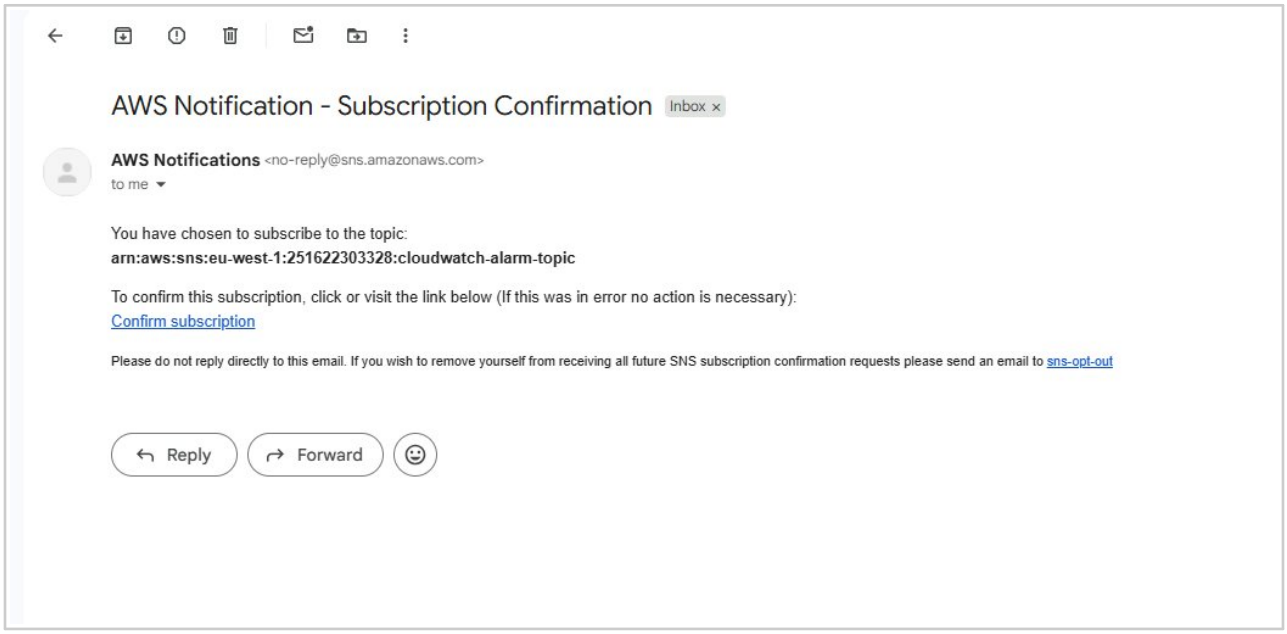


Figure 2.3 - AWS Subscription Confirmation email received in Gmail

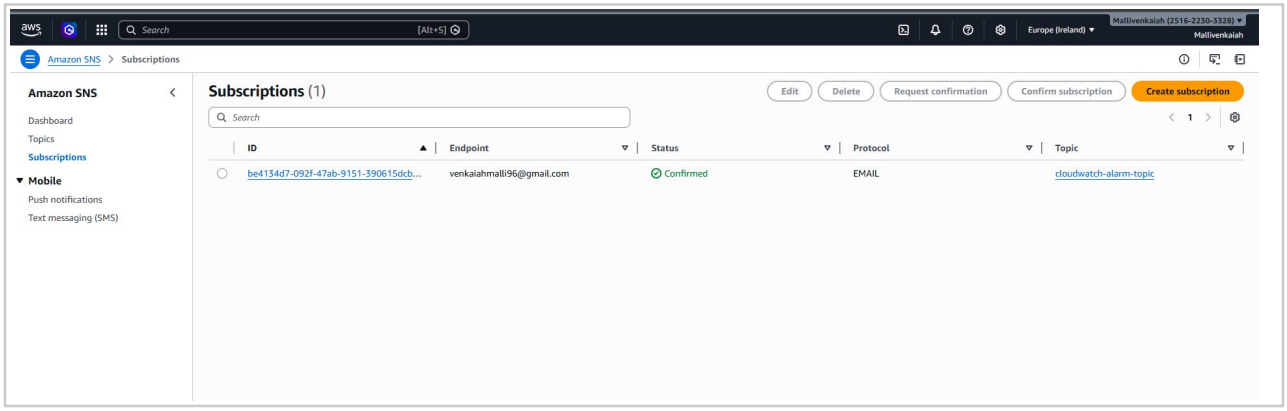


Figure 2.4 - SNS subscription status: Confirmed - ready to receive alarm notifications

3 Create CloudWatch Alarm

Configure the alarm to monitor EC2 CPU and trigger at 50% threshold

Now for the main part - creating the alarm. This is what does the actual monitoring. Once it is set up, it checks the CPU every minute on its own. I do not need to do anything - if the CPU goes above 50%, it fires the notification automatically.

3.1 - Select EC2 CPU Metric

I opened CloudWatch, went to Alarms, and clicked Create alarm. From there I clicked Select metric, chose EC2, then Per-Instance Metrics. I searched for my instance ID and selected the CPUUtilization row.

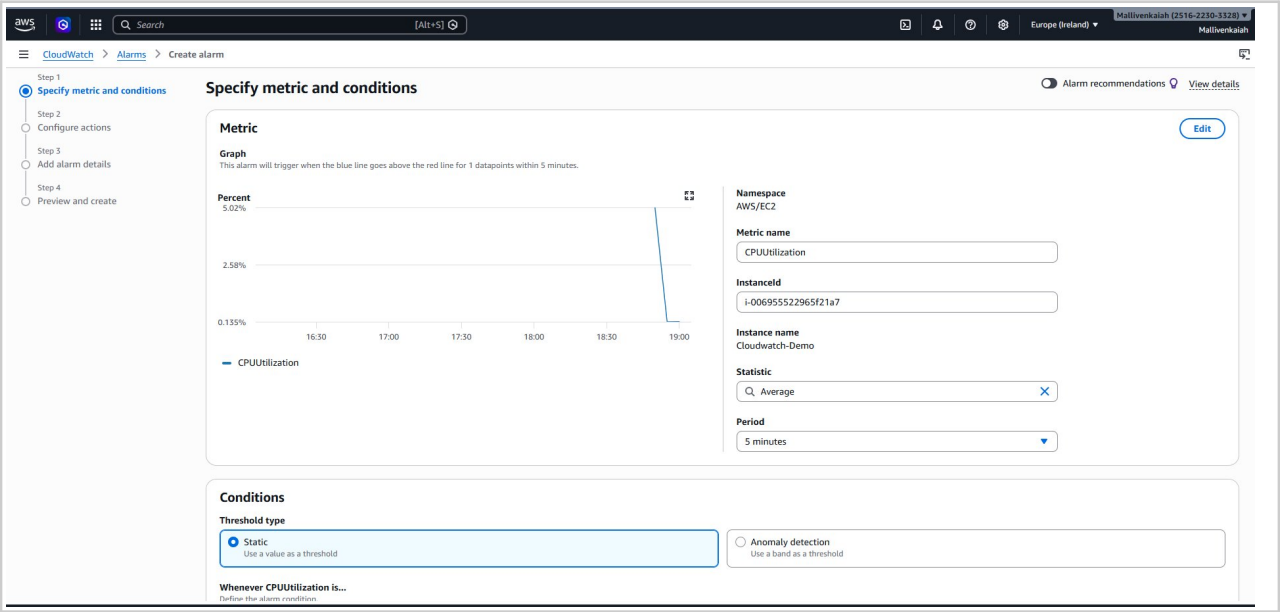


Figure 3.1 - CPUUtilization metric selected for instance Cloudwatch-Demo

3.2 - Set Alarm Conditions

Period	1 minute - CloudWatch evaluates CPU every 60 seconds (default is 5 minutes, changed for faster detection)
Statistic	Average - mean CPU across all data points within the 1-minute window
Threshold Type	Static - compare against a fixed value (vs Anomaly Detection which uses ML-based dynamic thresholds)
Condition	Greater than - alarm fires when CPU exceeds (strictly above) the threshold value
Threshold Value	50 - alarm fires when average CPU is above 50% for 1 consecutive minute
Datapoints to Alarm	1 out of 1 - alarm fires after first breach (use 3/3 in production to reduce false alarms)

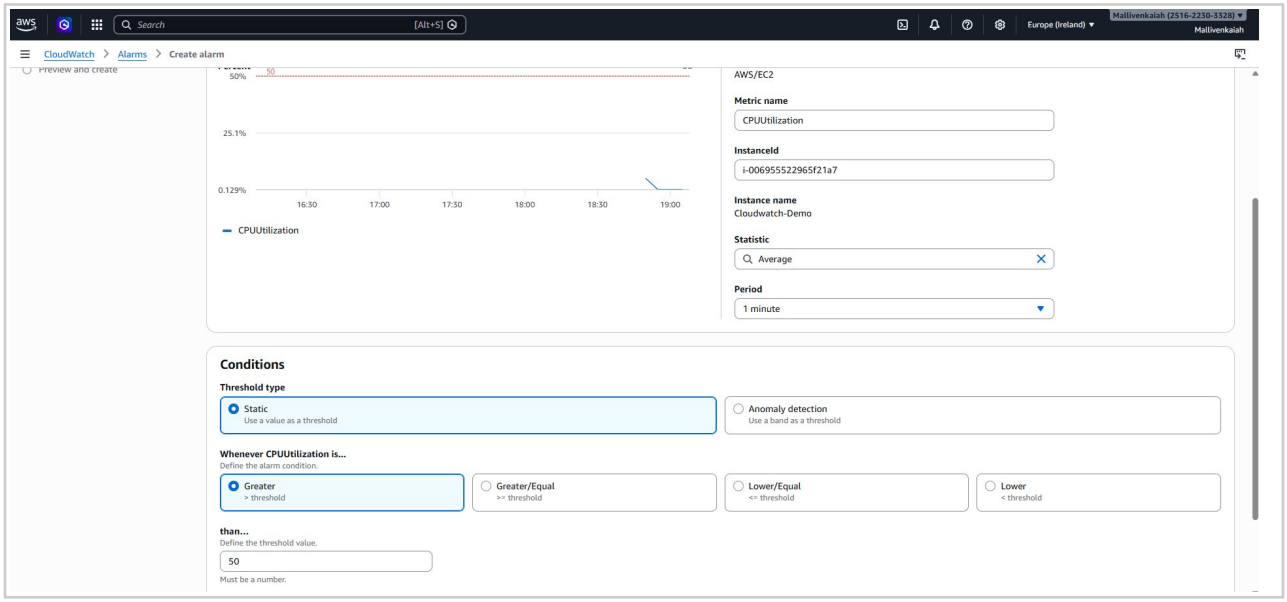


Figure 3.2 - Alarm conditions: CPUUtilization > 50 | Period: 1 minute | Statistic: Average

3.3 - Configure SNS Notification Action

On the next page I set the trigger to In alarm and selected the cloudwatch-alarm-topic I had already created. My confirmed email address appeared underneath it automatically.

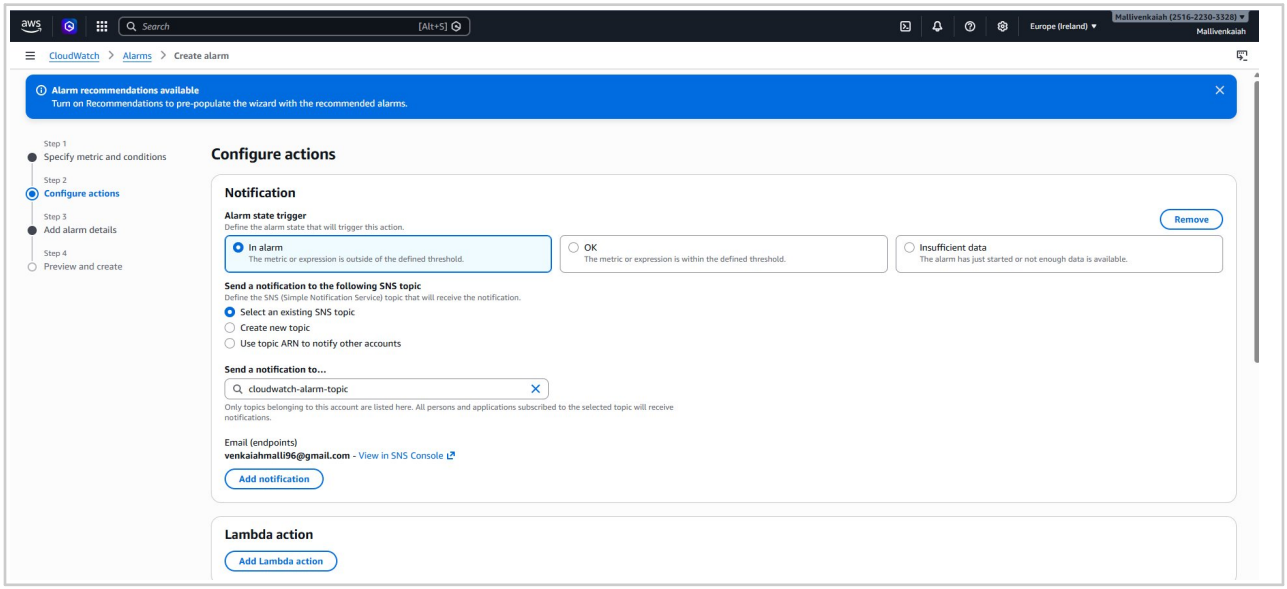


Figure 3.3 - SNS action configured: In alarm state triggers cloudwatch-alarm-topic

3.4 - Alarm Name & Creation

Alarm Name	CPU-High_Alert
Description	Alert when CPU exceeds 50 percent

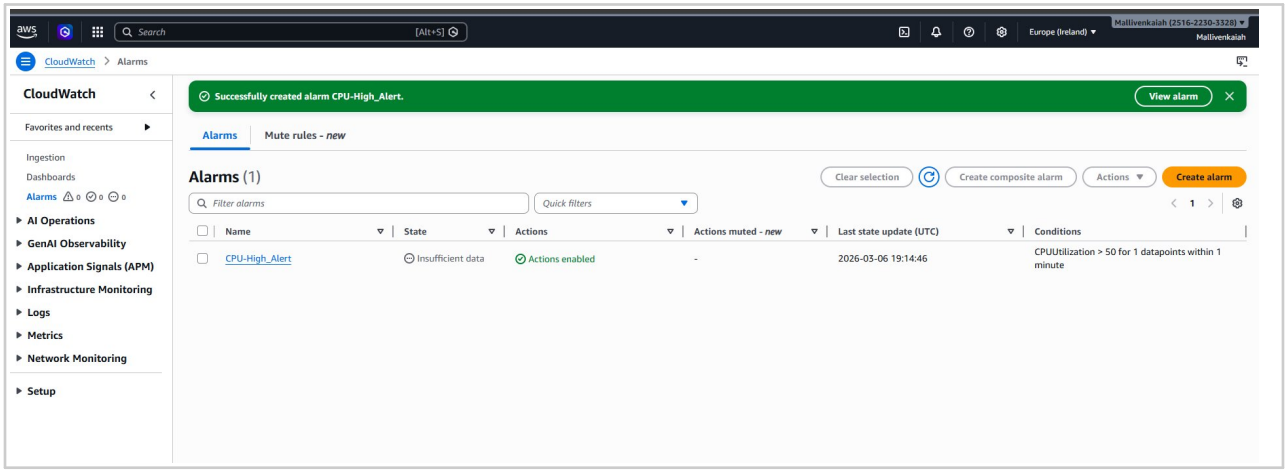


Figure 3.4 - CloudWatch alarm CPU-High_Alert created - initial state: Insufficient data

> **Note:** Insufficient data is the expected initial state after alarm creation. CloudWatch needs at least one 1-minute data point before it can evaluate the condition. The alarm transitions to OK within 1-2 minutes once data collection begins.

4 Connect to EC2 & Run Stress Test

Install the stress tool and push CPU above 50% to trigger the alarm

Now I needed to actually push the CPU above 50% to see if the alarm works. I connected to the instance using EC2 Instance Connect, installed the stress tool, and ran it. Within about 2 minutes CloudWatch picked up the spike and the alarm fired.

Connect via EC2 Instance Connect

I went to EC2, clicked on the Cloudwatch-Demo instance, and hit the Connect button. I chose the EC2 Instance Connect tab and clicked Connect. A terminal opened right in the browser - I did not need my key file or any local SSH tool.

Install Stress Tool

I ran the following command to install stress:

```
sudo yum install stress -y
```

Figure 4.1 - Stress tool installed: stress 1.0.7 from Amazon Linux repository

Run Stress Command

Execute to generate CPU load:

```
stress -c 2 -t 300
```

Parameter	Explanation
-c 2	Spawn 2 CPU stress workers - one per vCPU on t3.micro. Each maxes out a CPU core immediately.
-t 300	Run for 300 seconds (5 minutes). CloudWatch checks every minute - 5 minutes ensures detection and alarm confirmation.
Result	Both vCPUs reach ~100% immediately. CloudWatch measures average CPUUtilization at 99.93% after 1 minute.

```
[ec2-user@ip-172-31-47-189 ~]$ stress -c 2 -t 300
stress: info: [26026] dispatching hogs: 2 cpu, 0 io, 0 vm, 0 hdd
```

Figure 4.2 - Stress running: dispatching hogs: 2 cpu - both vCPUs under full load

5 Alarm Triggered - IN ALARM State

CloudWatch detects the CPU breach and transitions the alarm to IN ALARM

About 2 minutes after starting the stress test, the alarm changed state. CloudWatch measured the CPU at 99.93% - way above the 50% threshold - and moved the alarm from OK to IN ALARM. The dashboard went red and the graph showed a sharp spike. SNS sent the email almost straight away.

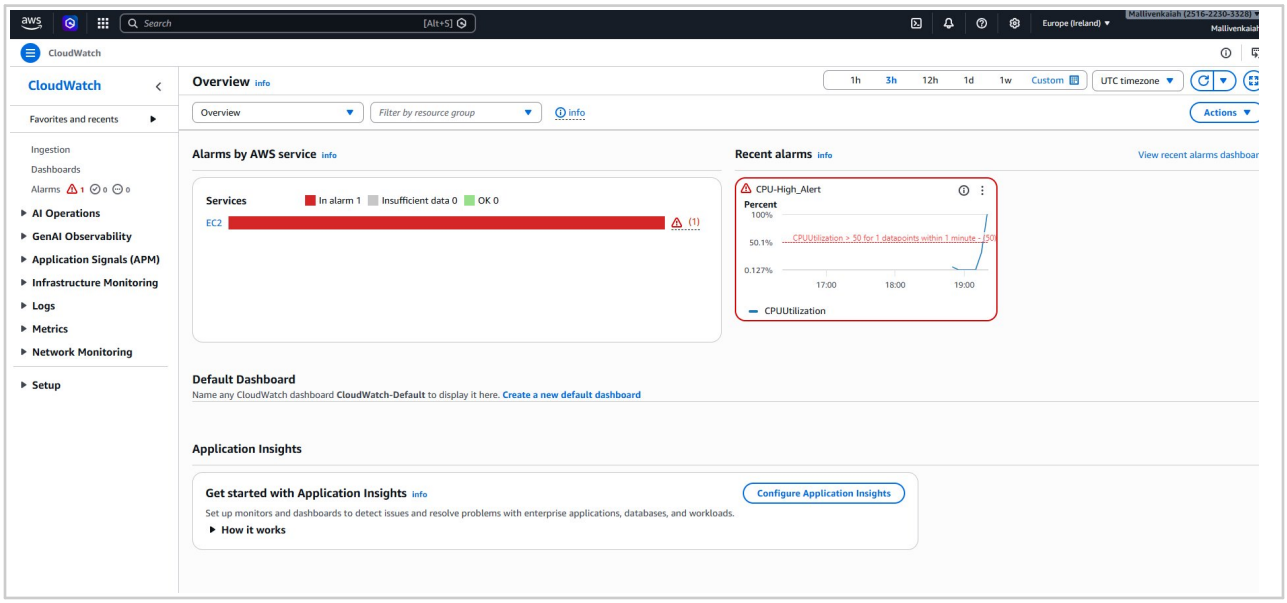


Figure 5.1 - CloudWatch Overview: CPU-High_Alert IN ALARM - EC2 showing 1 active alarm

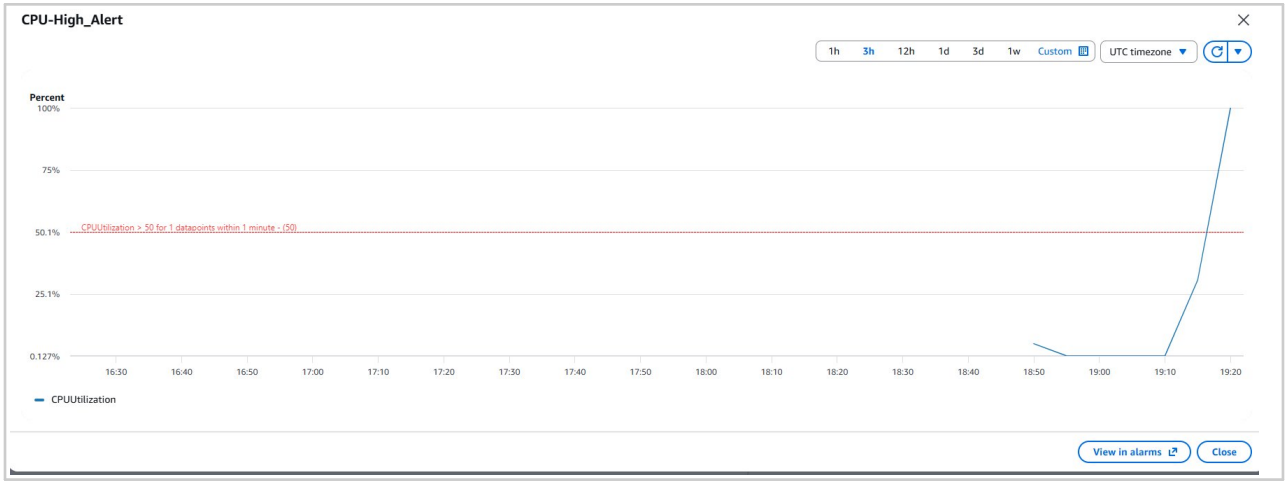


Figure 5.2 - CPU utilisation graph: line spikes above the 50% threshold - breach clearly visible

Alarm State Reference

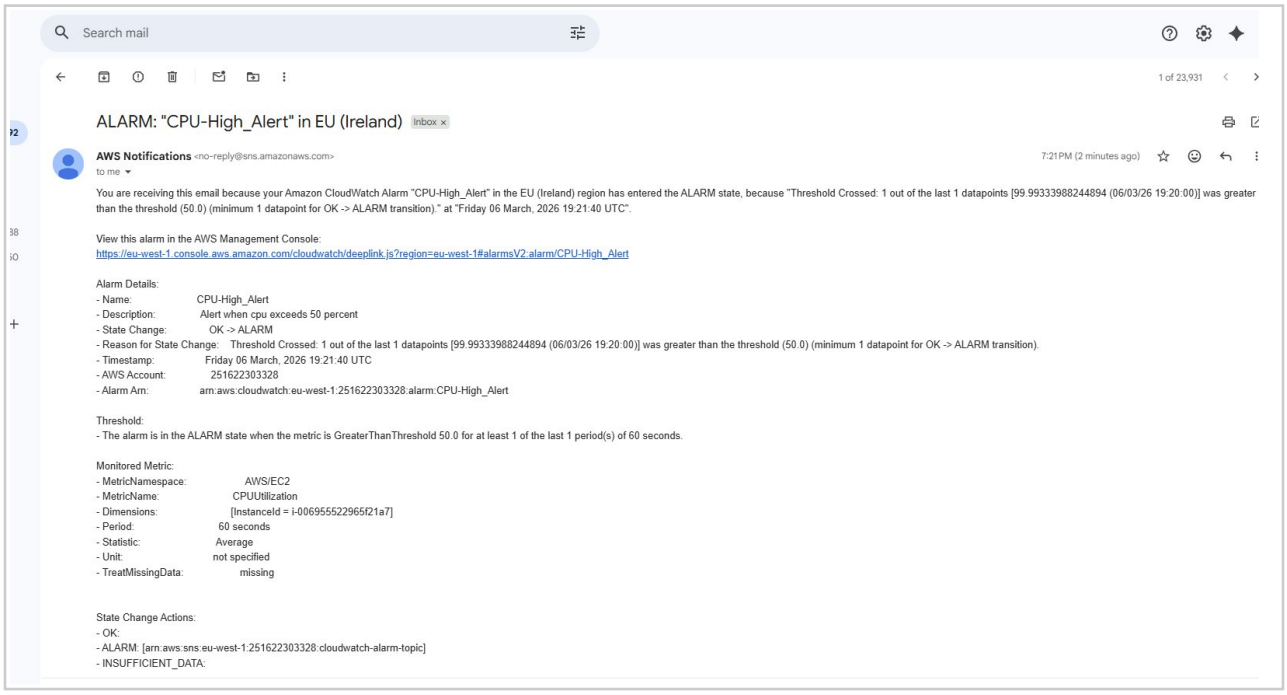
State	Meaning
INSUFFICIENT DATA	This was the state right after I created the alarm. CloudWatch had not collected any data yet so it could not evaluate anything.
OK	CPU is below 50% - the instance is running normally. No action is taken in this state.

IN ALARM	CPU crossed 50% for 1 consecutive minute. The alarm fired and SNS sent the notification to my Gmail.
What I saw	In my test the alarm moved from OK to IN ALARM at 19:21:40 UTC on 06 March 2026 - about 2 minutes after I started the stress test.

6 Email Notification Received

Amazon SNS delivers the alarm notification to Gmail within 2 minutes

The email arrived in my Gmail about a minute after the alarm fired. It came from AWS Notifications and had everything I needed - the exact CPU value that triggered it, the timestamp, the instance ID, and a direct link to the alarm in the console. Exactly what you would want to see if this were a real production incident.



Search mail

1 of 23,931

ALARM: "CPU-High_Alert" in EU (Ireland) Inbox ×

AWS Notifications <no-reply@sns.amazonaws.com> 7:21 PM (2 minutes ago)
to me

You are receiving this email because your Amazon CloudWatch Alarm "CPU-High_Alert" in the EU (Ireland) region has entered the ALARM state, because "Threshold Crossed: 1 out of the last 1 datapoints [99.99333988244894 (06/03/26 19:20:00)] was greater than the threshold (50.0) (minimum 1 datapoint for OK -> ALARM transition)." at "Friday 06 March, 2026 19:21:40 UTC".

View this alarm in the AWS Management Console:
https://eu-west-1.console.aws.amazon.com/cloudwatch/deeplink.js?region=eu-west-1#alarmsV2:alarm/CPU-High_Alert

Alarm Details:
- Name: CPU-High_Alert
- Description: Alert when cpu exceeds 50 percent
- State Change: OK -> ALARM
- Reason for State Change: Threshold Crossed: 1 out of the last 1 datapoints [99.99333988244894 (06/03/26 19:20:00)] was greater than the threshold (50.0) (minimum 1 datapoint for OK -> ALARM transition).
- Timestamp: Friday 06 March, 2026 19:21:40 UTC
- AWS Account: 251622303328
- Alarm Arn: arn:aws:cloudwatch:eu-west-1:251622303328:alarm:CPU-High_Alert

Threshold:
- The alarm is in the ALARM state when the metric is GreaterThanThreshold 50.0 for at least 1 of the last 1 period(s) of 60 seconds.

Monitored Metric:
- MetricNamespace: AWS/EC2
- MetricName: CPUUtilization
- Dimensions: [InstanceId = i-006955522965f21a7]
- Period: 60 seconds
- Statistic: Average
- Unit: not specified
- TreatMissingData: missing

State Change Actions:
- OK:
- ALARM: [arn:aws:sns:eu-west-1:251622303328:cloudwatch-alarm-topic]
- INSUFFICIENT_DATA:

Figure 6.1 - ALARM email received: Subject: ALARM: "CPU-High_Alert" in EU (Ireland)

Email Content

Subject	ALARM: "CPU-High_Alert" in EU (Ireland)
Sender	AWS Notifications
State Change	OK → ALARM
CPU Value at Breach	99.93% (1 out of last 1 datapoints exceeded the 50.0 threshold)
Timestamp	Friday 06 March 2026 - 19:21:40 UTC
Instance ID	i-006955522965f21a7 (Cloudwatch-Demo)
Metric	CPUUtilization - Namespace: AWS/EC2 - Period: 60s - Statistic: Average
AWS Account	251622303328 - Region: eu-west-1
Alarm ARN	arn:aws:cloudwatch:eu-west-1:251622303328:alarm:CPU-High_Alert

7 Resource Cleanup

Delete all AWS resources immediately to maintain zero cost

Once I confirmed everything worked, I deleted all the resources straight away. This project stayed within the Free Tier so the cost was zero, but I always clean up immediately after any lab or project. In a real work environment, a forgotten running instance can quietly add up to a large bill by the end of the month.

Resource	Steps to Delete
EC2 Instance	EC2 → Instances → Select Cloudwatch-Demo → Instance state → Terminate instance
CloudWatch Alarm	CloudWatch → Alarms → Select CPU-High_Alert → Actions → Delete
SNS Topic	SNS → Topics → Select cloudwatch-alarm-topic → Delete
Key Pair	EC2 → Network & Security → Key Pairs → Select cloudwatch-key → Delete

Free Tier Cost Breakdown

Service	Free Tier Limit	Used in This Project	Cost
EC2 t3.micro	750 hours/month	~1 hour	**€0.00**
CloudWatch Alarms	10 alarms/month	1 alarm	**€0.00**
SNS Notifications	1,000/month	2 emails	**€0.00**
Total			**€0.00**

5. Skills Demonstrated

I built this entire project through the AWS Console on a real account in eu-west-1. No scripts, no Terraform, no automation - just hands on the console. The instance ran, CPU spiked, alarm fired, email arrived. Total cost was zero. Here is what I practised.

Skill	What Was Done
AWS EC2	Launched a t3.micro on Amazon Linux 2023, configured the security group and key pair, connected via EC2 Instance Connect from the browser, and terminated it cleanly afterwards.
AWS CloudWatch	Created a CPU alarm with a 1-minute period and 50% threshold. Watched it go through Insufficient data when first created, OK once the instance was running, then IN ALARM when the stress test pushed CPU to 99.93%.
Amazon SNS	Created a Standard topic, added my Gmail as a subscriber, confirmed the subscription, and received the alarm email within a minute of the CPU crossing the threshold.
Linux Administration	Installed stress using yum on Amazon Linux 2023, ran the load test with two CPU workers for 5 minutes, and watched the CPU spike in the CloudWatch graph in near real time.
Cloud Monitoring	Set up a complete monitoring and alerting pipeline from scratch using only the AWS Console. Verified it end to end by triggering the alarm and confirming the email arrived.
Cost Management	Everything ran within the Free Tier. I deleted the instance, alarm, topic, and key pair immediately after the test. Total cost: zero.

Venkaiah Malli venkaiahmalli96@gmail.com github.com/venkaiahmalli96 Dublin, Ireland